

Brain Tumor Classification Using ResNet18 with PyTorch

May 14, 2026

1 Introduction

Brain tumor detection and classification from Magnetic Resonance Imaging (MRI) scans is a critical task in medical diagnostics. Accurate and timely classification can significantly improve treatment planning and patient outcomes. This project implements a deep learning-based classification system to automatically categorize brain MRI images into four classes: Glioma, Meningioma, No Tumor, and Pituitary tumor. The model utilizes ResNet18, a pre-trained convolutional neural network, fine-tuned on a publicly available brain tumor MRI dataset. The implementation leverages PyTorch for model training, data augmentation techniques for improved generalization, and comprehensive evaluation metrics.

1.1 Objectives:

- Develop a robust CNN-based classifier for multi-class brain tumor detection.
- Apply transfer learning to achieve high accuracy with limited medical data.
- Evaluate model performance using accuracy, loss curves, confusion matrix, and classification report.
- Provide a reproducible pipeline for medical image classification.

2 Background

2.1 Brain Tumor Types

- **Glioma:** Most common and aggressive primary brain tumor.
- **Meningioma:** Typically benign, arises from meninges.
- **Pituitary Tumor:** Affects the pituitary gland, can be hormone-secreting.
- **No Tumor:** Normal brain MRI scans.

Manual interpretation of MRI scans is time-consuming and subject to inter-observer variability. Deep learning, particularly Convolutional Neural Networks (CNNs), has shown remarkable success in medical image analysis due to their ability to automatically learn hierarchical features.

2.2 Transfer Learning

ResNet18, pre-trained on ImageNet, is used as the backbone. Transfer learning allows the model to leverage low-level features (edges, textures) learned from natural images and adapt them to medical images, significantly reducing training time and data requirements.

```
[52]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import numpy as np
import os
from PIL import Image
import pandas as pd
import seaborn as sns
from glob import glob
from sklearn.metrics import classification_report, confusion_matrix

[2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

3 Methodology

3.1 Dataset

- **Source:** Kaggle Brain Tumor MRI Dataset (archive folder).
- **Training Set:** 5,712 images.
- **Testing Set:** 1,311 images.
- **Classes:** 4 (Glioma, Meningioma, Notumor, Pituitary).
- Class distribution is reasonably balanced.

```
[3]: batch_size = 256
learning_rate = 0.001
num_epochs = 20
image_size = 256
```

3.2 Data Preprocessing & Augmentation

Training Transformations: - Resize to 256×256. - RandomResizedCrop, Horizontal Flip, Rotation ($\pm 15^\circ$). - ColorJitter (brightness, contrast, saturation, hue). - Normalization using ImageNet statistics.

Testing Transformations: Resize + Normalization only.

```
[4]: train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomResizedCrop(image_size, scale=(0.8, 1.0)),    # zoom-like
    ↪ effect
    transforms.RandomHorizontalFlip(p=0.5),
```

```

        transforms.RandomRotation(degrees=15),                                # rotation /
    ↪tilt
        transforms.ColorJitter(brightness=0.2, contrast=0.2,
                                saturation=0.2, hue=0.05),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],                    # ImageNet
    ↪normalization
                                std=[0.229, 0.224, 0.225])
    ])

test_transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

```

```

[5]: def train_df(tr_path):
        classes, class_paths = zip(*[(label, os.path.join(tr_path, label, image))
    ↪for label in os.listdir(tr_path) if os.path.
    ↪isdir(os.path.join(tr_path, label))
    ↪for image in os.listdir(os.path.join(tr_path,
    ↪label))])

        tr_df = pd.DataFrame({'Class Path': class_paths, 'Class': classes})
        return tr_df

def test_df(ts_path):
        classes, class_paths = zip(*[(label, os.path.join(ts_path, label, image))
    ↪for label in os.listdir(ts_path) if os.path.
    ↪isdir(os.path.join(ts_path, label))
    ↪for image in os.listdir(os.path.join(ts_path,
    ↪label))])

        ts_df = pd.DataFrame({'Class Path': class_paths, 'Class': classes})
        return ts_df

```

```

[8]: VALID_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.tif', '.tiff')

def train_df(tr_path):
    rows = []

    for label in os.listdir(tr_path):
        class_dir = os.path.join(tr_path, label)

        if not os.path.isdir(class_dir):

```

```

        continue

    for file_name in os.listdir(class_dir):
        file_path = os.path.join(class_dir, file_name)

        if os.path.isfile(file_path) and file_name.lower().
↪endswith(VALID_EXTENSIONS):
            rows.append((file_path, label))

    return pd.DataFrame(rows, columns=['Class Path', 'Class'])

def test_df(ts_path):
    rows = []

    for label in os.listdir(ts_path):
        class_dir = os.path.join(ts_path, label)

        if not os.path.isdir(class_dir):
            continue

        for file_name in os.listdir(class_dir):
            file_path = os.path.join(class_dir, file_name)

            if os.path.isfile(file_path) and file_name.lower().
↪endswith(VALID_EXTENSIONS):
                rows.append((file_path, label))

    return pd.DataFrame(rows, columns=['Class Path', 'Class'])

```

```
[9]: tr_df = train_df('archive/Training')
```

```
[10]: tr_df
```

```
[10]:
```

	Class Path	Class
0	archive/Training/glioma/Tr-glTr_0000.jpg	glioma
1	archive/Training/glioma/Tr-glTr_0001.jpg	glioma
2	archive/Training/glioma/Tr-glTr_0002.jpg	glioma
3	archive/Training/glioma/Tr-glTr_0003.jpg	glioma
4	archive/Training/glioma/Tr-glTr_0004.jpg	glioma
...	...	...
5707	archive/Training/pituitary/Tr-pi_1452.jpg	pituitary
5708	archive/Training/pituitary/Tr-pi_1453.jpg	pituitary
5709	archive/Training/pituitary/Tr-pi_1454.jpg	pituitary
5710	archive/Training/pituitary/Tr-pi_1455.jpg	pituitary
5711	archive/Training/pituitary/Tr-pi_1456.jpg	pituitary

[5712 rows x 2 columns]

```
[11]: ts_df = train_df('archive/Testing')
```

```
[12]: ts_df
```

```
[12]:
```

	Class Path	Class
0	archive/Testing/glioma/Te-glTr_0000.jpg	glioma
1	archive/Testing/glioma/Te-glTr_0001.jpg	glioma
2	archive/Testing/glioma/Te-glTr_0002.jpg	glioma
3	archive/Testing/glioma/Te-glTr_0003.jpg	glioma
4	archive/Testing/glioma/Te-glTr_0004.jpg	glioma
...	...	...
1306	archive/Testing/pituitary/Te-pi_0295.jpg	pituitary
1307	archive/Testing/pituitary/Te-pi_0296.jpg	pituitary
1308	archive/Testing/pituitary/Te-pi_0297.jpg	pituitary
1309	archive/Testing/pituitary/Te-pi_0298.jpg	pituitary
1310	archive/Testing/pituitary/Te-pi_0299.jpg	pituitary

[1311 rows x 2 columns]

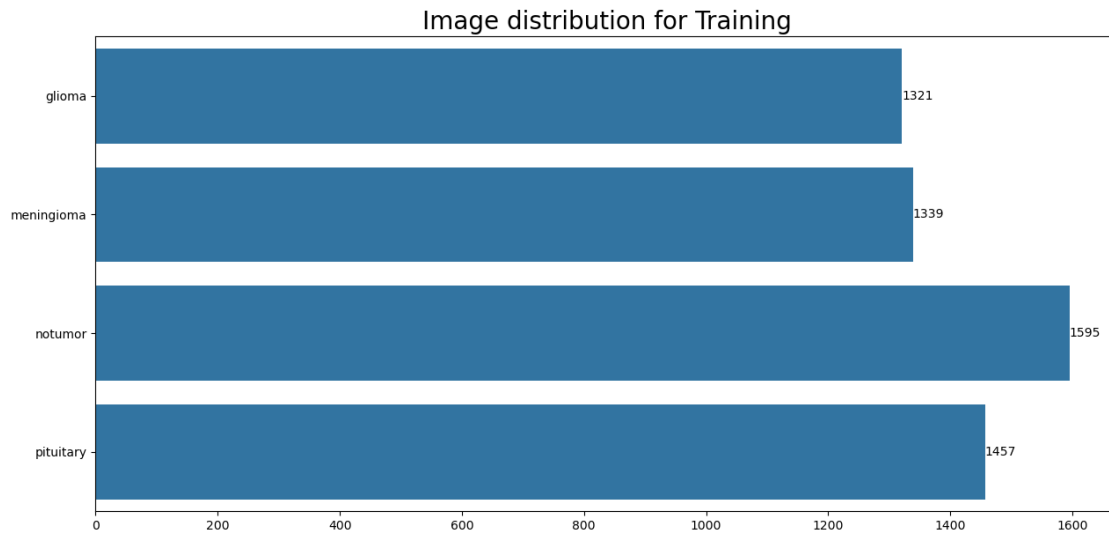
```
[13]: print(tr_df.head())
print(tr_df[~tr_df['Class Path'].apply(os.path.isfile)])
```

	Class Path	Class
0	archive/Training/glioma/Tr-glTr_0000.jpg	glioma
1	archive/Training/glioma/Tr-glTr_0001.jpg	glioma
2	archive/Training/glioma/Tr-glTr_0002.jpg	glioma
3	archive/Training/glioma/Tr-glTr_0003.jpg	glioma
4	archive/Training/glioma/Tr-glTr_0004.jpg	glioma

Empty DataFrame
Columns: [Class Path, Class]
Index: []

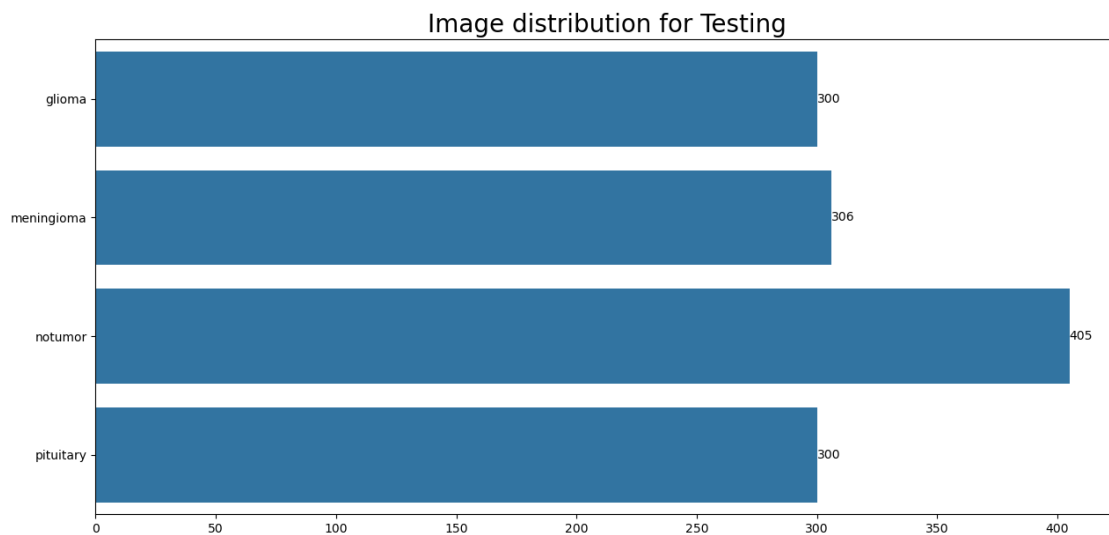
```
[14]: plt.figure(figsize=(15,7))
ax = sns.countplot(data=tr_df , y=tr_df['Class'])

plt.xlabel('')
plt.ylabel('')
plt.title('Image distribution for Training', fontsize=20)
ax.bar_label(ax.containers[0])
plt.show()
```



```
[15]: plt.figure(figsize=(15,7))
ax = sns.countplot(data=ts_df , y=ts_df['Class'])

plt.xlabel('')
plt.ylabel('')
plt.title('Image distribution for Testing', fontsize=20)
ax.bar_label(ax.containers[0])
plt.show()
```



```
[16]: all_classes = sorted(tr_df['Class'].unique())
      class_to_idx = {cls_name: idx for idx, cls_name in enumerate(all_classes)}

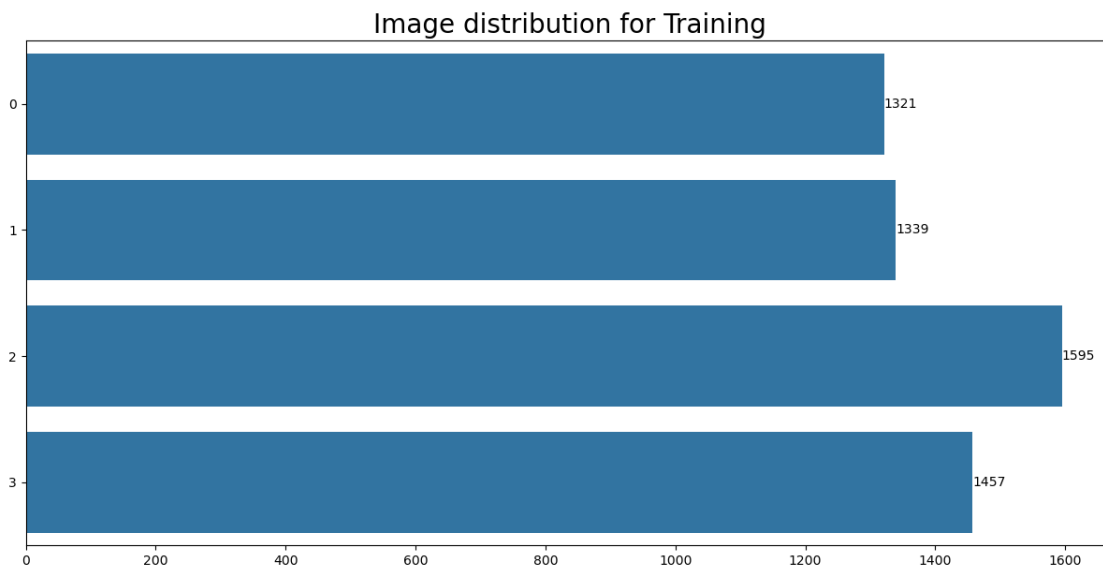
      print(class_to_idx)
```

```
{'glioma': 0, 'meningioma': 1, 'notumor': 2, 'pituitary': 3}
```

```
[17]: tr_df['Label'] = tr_df['Class'].map(class_to_idx)
      ts_df['Label'] = ts_df['Class'].map(class_to_idx)
```

```
[18]: plt.figure(figsize=(15,7))
      ax = sns.countplot(data=tr_df , y=tr_df['Label'])

      plt.xlabel('')
      plt.ylabel('')
      plt.title('Image distribution for Training', fontsize=20)
      ax.bar_label(ax.containers[0])
      plt.show()
```



```
[19]: from torch.utils.data import Dataset

      class TumorDataset(Dataset):
          def __init__(self, df, transform=None):
              self.df = df.reset_index(drop=True)
              self.transform = transform
              self.label = sorted(df['Label'].unique().tolist())

          def __len__(self):
```

```

        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.loc[idx, 'Class Path']
        label = self.df.loc[idx, 'Label']

        if not os.path.isfile(img_path):
            raise FileNotFoundError(f"Not a valid file: {img_path}")

        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, torch.tensor(label, dtype=torch.long)

```

```

[20]: train_dataset = TumorDataset(tr_df, transform=train_transform)
      test_dataset = TumorDataset(ts_df, transform=test_transform)

```

3.3 Model Architecture

- **Base Model:** ResNet18 (pre-trained).
- **Modification:** Replaced the final fully connected layer with a new Linear layer outputting 4 classes.
- **Optimizer:** Adam (lr=0.001).
- **Loss Function:** CrossEntropyLoss.
- **Learning Rate Scheduler:** StepLR (decay every 5 epochs).
- **Batch Size:** 32

```

[21]: from torch.utils.data import random_split

train_size = int(0.8 * len(train_dataset))
val_size   = len(train_dataset) - train_size

train_dataset, val_dataset = random_split(train_dataset, [train_size, val_size])

val_dataset.dataset.transform = test_transform

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size=32, shuffle=False)

class_names = train_dataset.dataset.label
num_classes = len(class_names)

print("Classes:", class_names)
print("Number of classes:", num_classes)

```


Classes: [0, 1, 2, 3]
Number of classes: 4

```
[22]: model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

```
# Replace final fully connected layer
model.fc = nn.Linear(model.fc.in_features, num_classes)

model = model.to(device)
print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
```

```

track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
)
(1): BasicBlock(
    (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
)
(layer3): Sequential(
    (0): BasicBlock(
        (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (downsample): Sequential(
            (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
            (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
    )
    (1): BasicBlock(
        (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,

```

```

1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    )
    (layer4): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (downsample): Sequential(
          (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
    (fc): Linear(in_features=512, out_features=4, bias=True)
  )

```

```

[23]: criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Optional learning rate scheduler
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

```

3.4 Training Strategy

- 80/20 train-validation split on training data.
- Early stopping based on best validation accuracy.

- Trained for up to 20 epochs.

```
[24]: def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    running_correct = 0
    total_samples = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, preds = torch.max(outputs, 1)
        running_correct += (preds == labels).sum().item()
        total_samples += labels.size(0)

    epoch_loss = running_loss / total_samples
    epoch_acc = running_correct / total_samples

    return epoch_loss, epoch_acc
```

```
[54]: def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_correct = 0
    total_samples = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)
            _, preds = torch.max(outputs, 1)
            running_correct += (preds == labels).sum().item()
```

```

        total_samples += labels.size(0)

    if loader == test_loader:
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

epoch_loss = running_loss / total_samples
epoch_acc = running_correct / total_samples

return epoch_loss, epoch_acc

```

```

[50]: import copy
best_val_acc = 0.0
best_model_wts = copy.deepcopy(model.state_dict())
tr_loss = []
tr_acc = []
va_loss = []
va_acc = []
ep = []

for epoch in range(num_epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, u
    ↪optimizer, device)
    val_loss, val_acc = evaluate(model, val_loader, criterion, device)

    scheduler.step()

    print(f"Epoch [{epoch+1}/{num_epochs}]")
    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc*100:.2f}%")
    print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc*100:.2f}%")
    print("-" * 50)

    tr_loss.append(train_loss)
    tr_acc.append(train_acc)
    va_loss.append(val_loss)
    va_acc.append(val_acc)
    ep.append(epoch+1)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model_wts = copy.deepcopy(model.state_dict())

# Load best model
model.load_state_dict(best_model_wts)
print(f"Best Validation Accuracy: {best_val_acc*100:.2f}%")

```

4 Results & Evaluation

4.1 Training Performance

The model demonstrated excellent convergence:

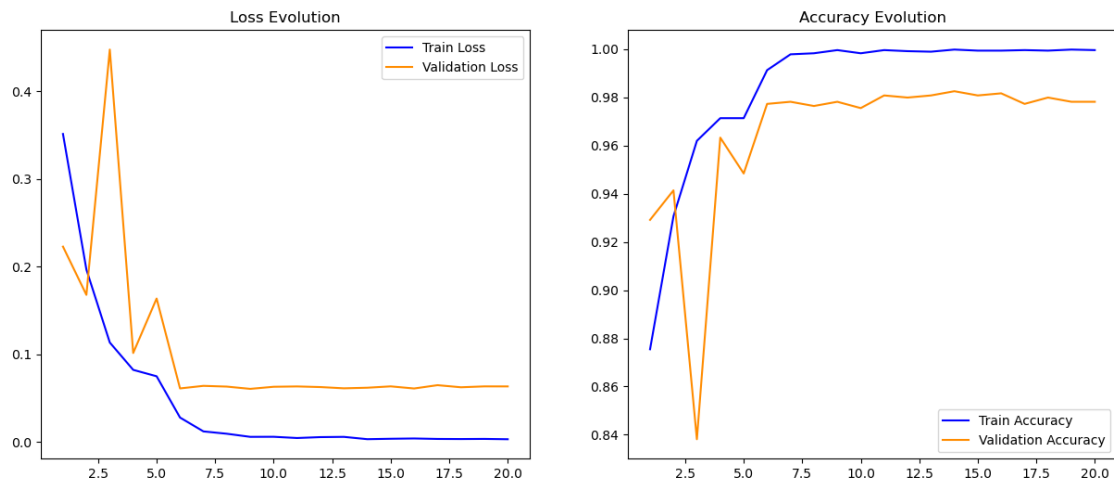
- **Final Test Accuracy:** 98.25%
- **Test Loss:** 0.0485

```
[27]: plt.figure(figsize=(15,6))

# Plotting the training and validation loss
plt.subplot(1, 2, 1)
plt.plot(ep, tr_loss, label='Train Loss', color='blue')
plt.plot(ep, va_loss, label='Validation Loss', color='darkorange')
plt.legend()
plt.title('Loss Evolution')

# Plotting the training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(ep, tr_acc, label='Train Accuracy', color='blue')
plt.plot(ep, va_acc, label='Validation Accuracy', color='darkorange')
plt.legend()
plt.title('Accuracy Evolution')

plt.show()
```



```
[55]: #model.load_state_dict(torch.load("resnet18_best_model.pth", ↵
      ↪map_location=device))
#model.to(device)
#model.eval()
y_true = []
```

```

y_pred = []
test_loss, test_acc = evaluate(model, test_loader, criterion, device)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc*100:.2f}%")

torch.save(model.state_dict(), "resnet18_best_model.pth")
print("Model saved as resnet18_best_model.pth")

```

Test Loss: 0.0485

Test Accuracy: 98.25%

Model saved as resnet18_best_model.pth

4.2 Confusion Matrix Analysis

The confusion matrix shows very strong performance:

- **No-tumor:** Near-perfect classification (405/405 correct).
- **Glioma:** 291 correct, minimal misclassifications.
- **Pituitary:** 298 correct.
- **Meningioma:** 294 correct, with very few confusions.

Minor confusion exists between Meningioma and other classes, which is clinically understandable due to overlapping imaging characteristics.

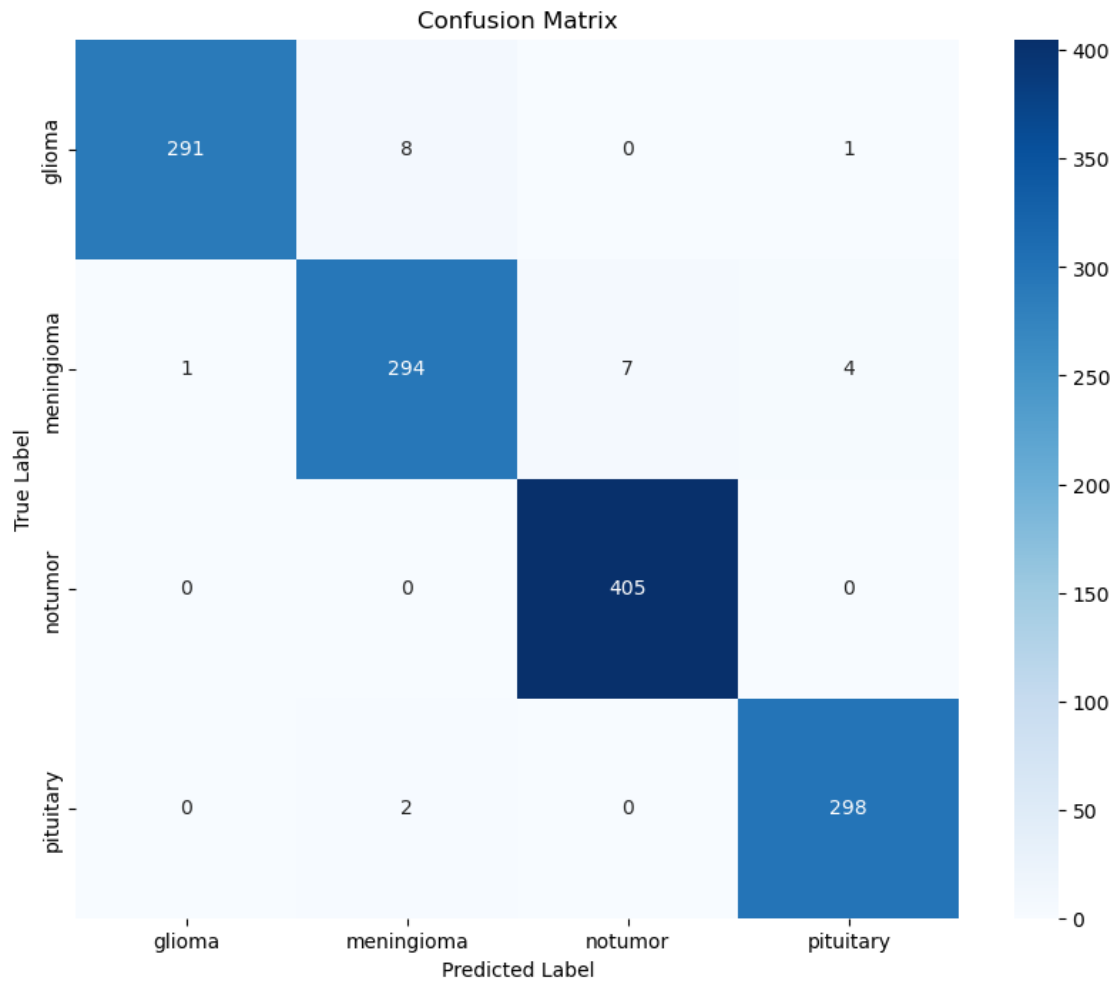
```

[57]: cm = confusion_matrix(y_true, y_pred)
      labels = list(class_to_idx.keys())

      plt.figure(figsize=(10,8))
      sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                  xticklabels=labels,
                  yticklabels=labels)

      plt.xlabel('Predicted Label')
      plt.ylabel('True Label')
      plt.title('Confusion Matrix')
      plt.show()

```



```
[43]: def predict(img_path):
    img = Image.open(img_path).convert("RGB")
    img.show()

    transform = transforms.Compose([
        transforms.Resize((256, 256)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                              [0.229, 0.224, 0.225])
    ])

    input_tensor = transform(img)
    input_tensor = input_tensor.unsqueeze(0)

    input_tensor = input_tensor.to(device)
```



```

with torch.no_grad():
    outputs = model(input_tensor)
    _, predicted = torch.max(outputs, 1)

pred_class = predicted.item()

if pred_class == 0:
    label_name = "glioma"
elif pred_class == 1:
    label_name = "meningioma"
elif pred_class == 2:
    label_name = "notumor"
elif pred_class == 3:
    label_name = "pituitary"

plt.imshow(img)
plt.title(f"Prediction: {label_name}")
plt.axis("off")
plt.show()

```

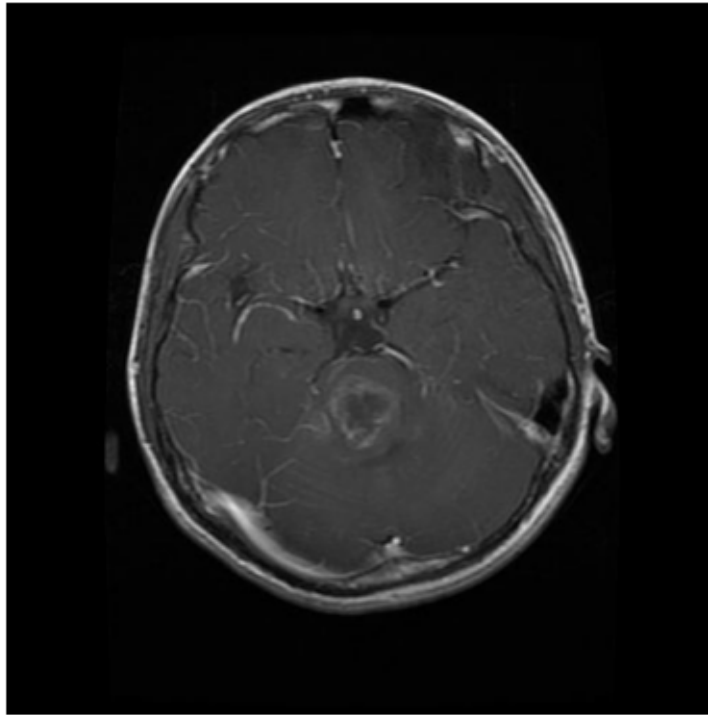
```
[44]: predict('archive/Testing/glioma/Te-glTr_0007.jpg')
```

```

/usr/bin/xdg-open: 882: www-browser: not found
/usr/bin/xdg-open: 882: links2: not found
/usr/bin/xdg-open: 882: elinks: not found
/usr/bin/xdg-open: 882: links: not found
/usr/bin/xdg-open: 882: lynx: not found
/usr/bin/xdg-open: 882: w3m: not found
xdg-open: no method available for opening '/tmp/tmpk4l1jmvn.PNG'

```

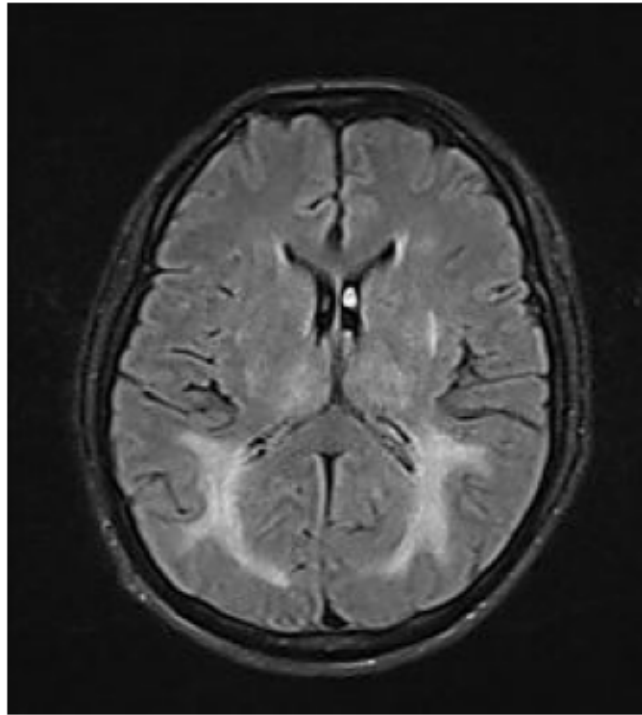
Prediction: glioma



```
[45]: predict('archive/Testing/notumor/Te-noTr_0001.jpg')
```

```
/usr/bin/xdg-open: 882: www-browser: not found  
/usr/bin/xdg-open: 882: links2: not found  
/usr/bin/xdg-open: 882: elinks: not found  
/usr/bin/xdg-open: 882: links: not found  
/usr/bin/xdg-open: 882: lynx: not found  
/usr/bin/xdg-open: 882: w3m: not found  
xdg-open: no method available for opening '/tmp/tmpz51frvx7.PNG'
```

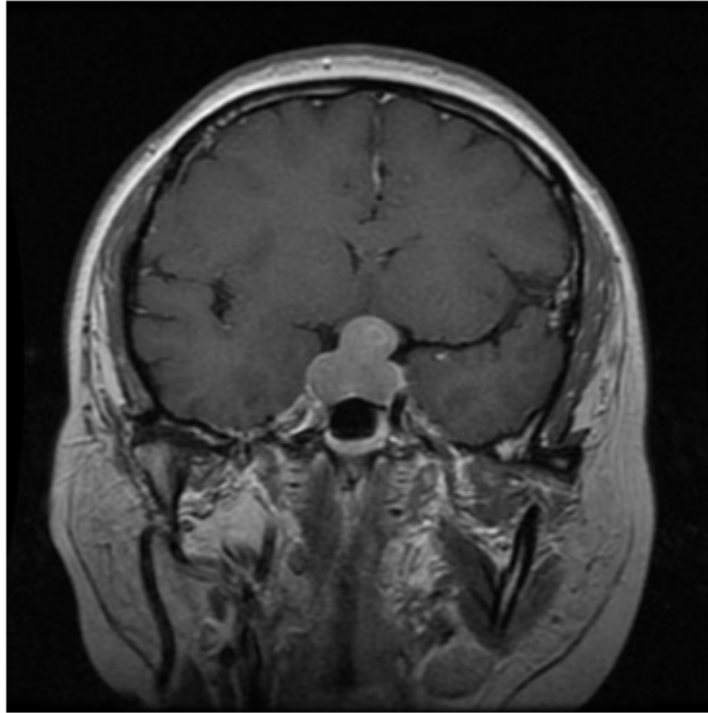
Prediction: notumor



```
[46]: predict('archive/Testing/pituitary/Te-piTr_0001.jpg')
```

```
/usr/bin/xdg-open: 882: www-browser: not found  
/usr/bin/xdg-open: 882: links2: not found  
/usr/bin/xdg-open: 882: elinks: not found  
/usr/bin/xdg-open: 882: links: not found  
/usr/bin/xdg-open: 882: lynx: not found  
/usr/bin/xdg-open: 882: w3m: not found  
xdg-open: no method available for opening '/tmp/tmpc7ul63_q.PNG'
```

Prediction: pituitary



5 Discussion

5.1 Strengths:

- High classification accuracy (98.25%) on the test set.
- Effective use of data augmentation to handle limited medical data.
- Good generalization (low gap between train and validation).
- Clean implementation with proper dataset handling and evaluation.

5.2 Limitations:

- Training was interrupted manually (KeyboardInterrupt).
- Model is relatively heavy (ResNet18) — lighter models (MobileNet, EfficientNet) could be explored for deployment.
- No cross-validation performed.
- Dataset may not represent real-world clinical variability (scanner types, protocols, patient demographics).

5.3 Potential Improvements:

- Ensemble learning.
- Attention mechanisms or Vision Transformers.
- Uncertainty quantification.

- Grad-CAM visualizations for explainability.
- Larger and more diverse datasets.

6 Conclusion

This project successfully demonstrates the application of transfer learning using ResNet18 for multi-class brain tumor classification from MRI images. Achieving 98.25% test accuracy highlights the effectiveness of modern deep learning techniques in medical imaging tasks.

The developed pipeline — from data loading and augmentation to model training and evaluation — is robust, reproducible, and can serve as a strong baseline for further research or clinical decision-support systems.

With additional refinements (explainability, deployment optimization, and external validation), this approach has strong potential for real-world assistive diagnostic applications.